

AR Vision

AR Vision

1. AR Introduction and Overview
2. Run Case
 - 2.1 Camera Intrinsic Parameter Calibration
 - 2.2 Program Startup
 - 2.3 RANSAC Solution
3. Source Code Analysis

1. AR Introduction and Overview

Augmented Reality (AR) is a technology that cleverly integrates virtual information with the real world. It extensively uses various technical means such as multimedia, 3D modeling, real-time tracking and registration, intelligent interaction, and sensing. After simulating virtual information such as computer-generated text, images, 3D models, music, and videos, it applies them to the real world. The two types of information complement each other, thereby achieving "enhancement" of the real world.

AR systems have three prominent features: ① Integration of real world and virtual world information; ② Real-time interactivity; ③ Adding and positioning virtual objects in 3D scale space.

Augmented reality technology includes new technologies and methods such as multimedia, 3D modeling, real-time video display and control, multi-sensor fusion, real-time tracking and registration, and scene fusion.

2. Run Case

When using AR cases, camera intrinsic parameters are required, otherwise it cannot run (the factory image has completed camera intrinsic parameter calibration). The intrinsic parameter file is in the same directory as the code.

2.1 Camera Intrinsic Parameter Calibration

[!IMPORTANT]

- The factory image has already performed camera calibration, so no calibration is needed for first use. If you need to calibrate yourself later, refer to this part of the tutorial. The camera calibration drawing is in the attachments of the supporting course materials and can be downloaded and printed by yourself.

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

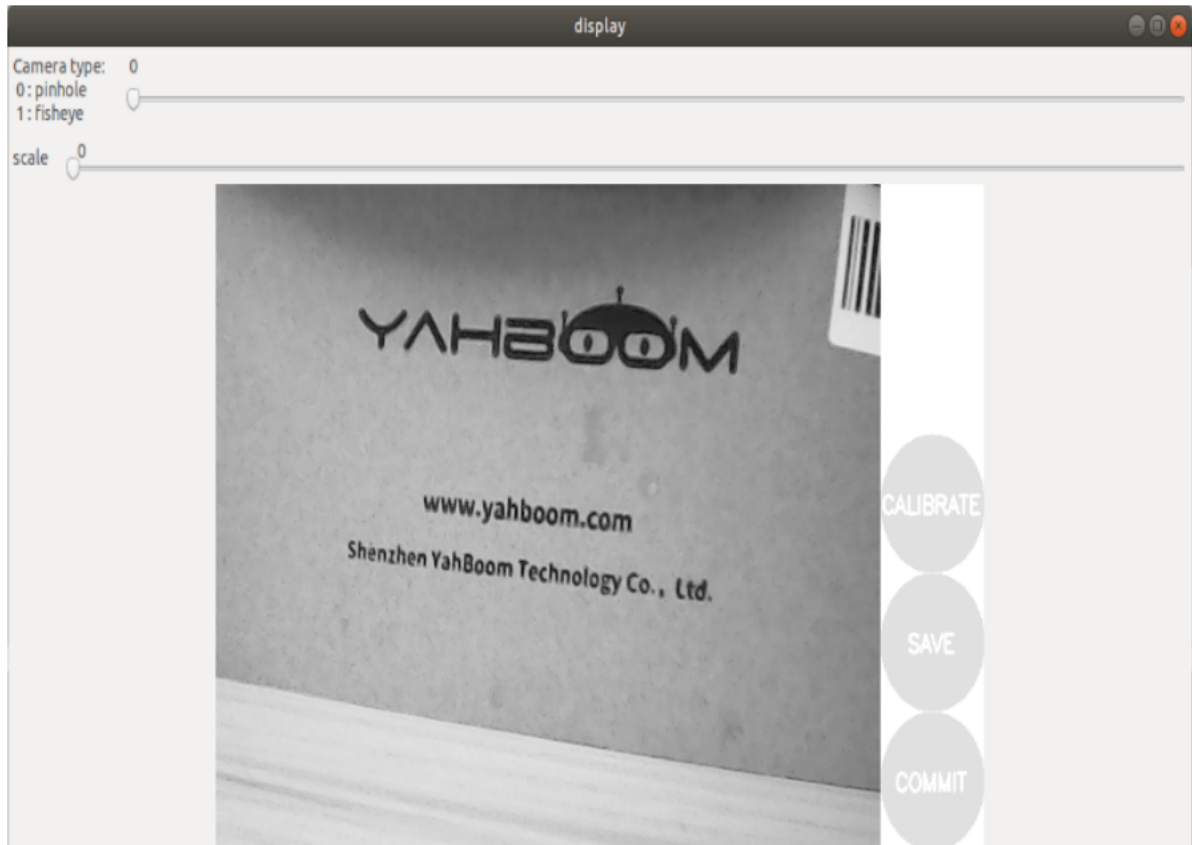
Open a terminal inside the container,

```
exec_m3
```

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=camera_calibration
```

size: The number of internal corner points of the calibration chessboard, for example 9X6, with a total of six rows and nine columns of corner points.

square: The side length of the chessboard, in meters.



Calibration interface

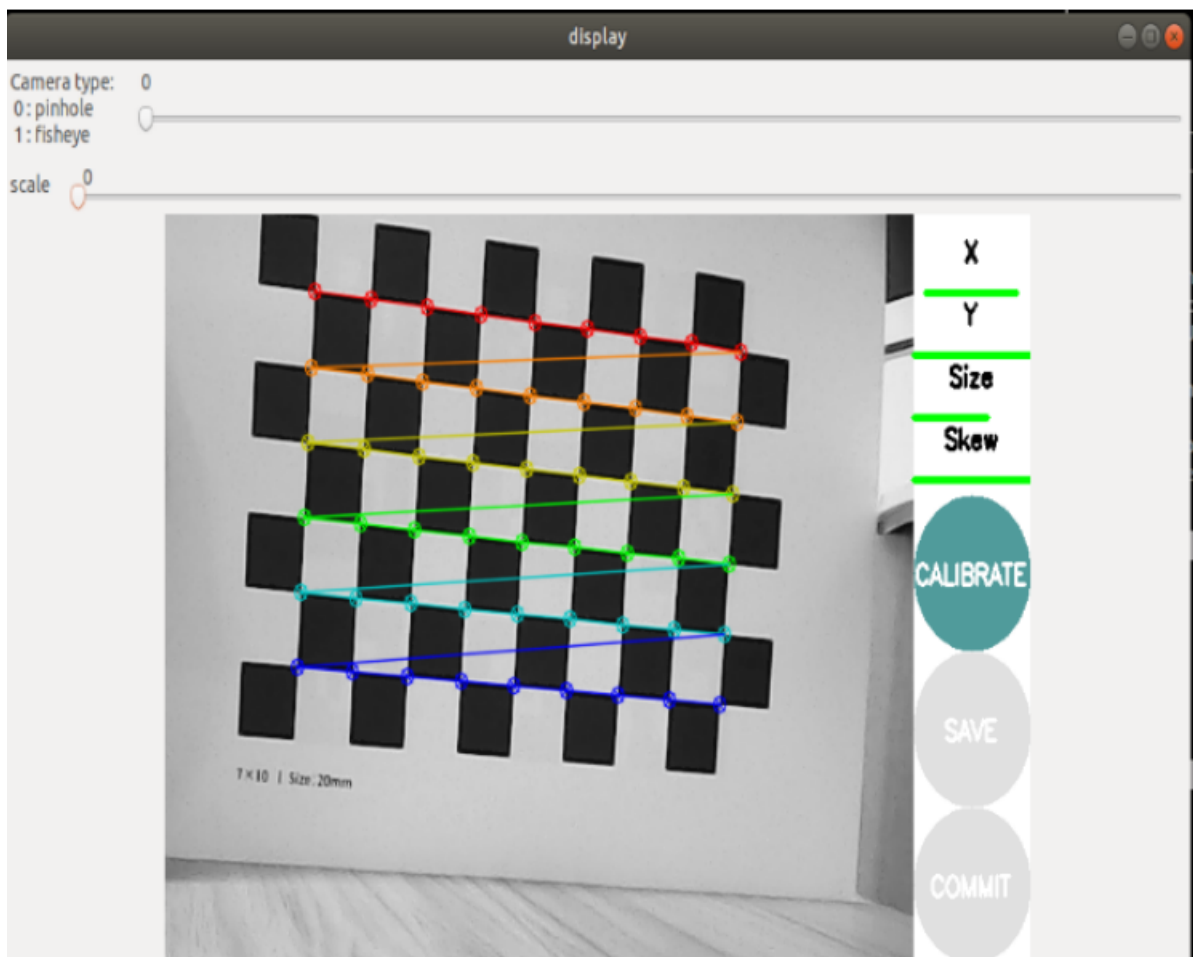
X: Left and right movement of the chessboard in the camera's field of view

Y: Up and down movement of the chessboard in the camera's field of view

Size: Front and back movement of the chessboard in the camera's field of view

Skew: Tilting rotation of the chessboard in the camera's field of view

After successful startup, place the chessboard in the center of the screen and change to different poses. The system will automatically recognize. The best situation is that the lines under **【X】**, **【Y】**, **【Size】**, **【Skew】** change from red to yellow to green as data is collected, try to fill them as much as possible.



- Click **【CALIBRATE】** to calculate camera intrinsic parameters. The more pictures, the longer it takes, just wait. (Sixty or seventy pictures are enough, too many may cause freezing).
- Click **【SAVE】** to save the calibration results to **【/tmp/calibrationdata.tar.gz】** in the current running terminal.
- After calibration, you can move the **【/tmp/calibrationdata.tar.gz】** file to the user directory to view the contents

```
sudo mv /tmp/calibrationdata.tar.gz ~
```

After decompression, there are the just-calibrated images, an ost.txt file and an ost.yaml file. ost.yaml is the intrinsic parameter of the calibrated camera

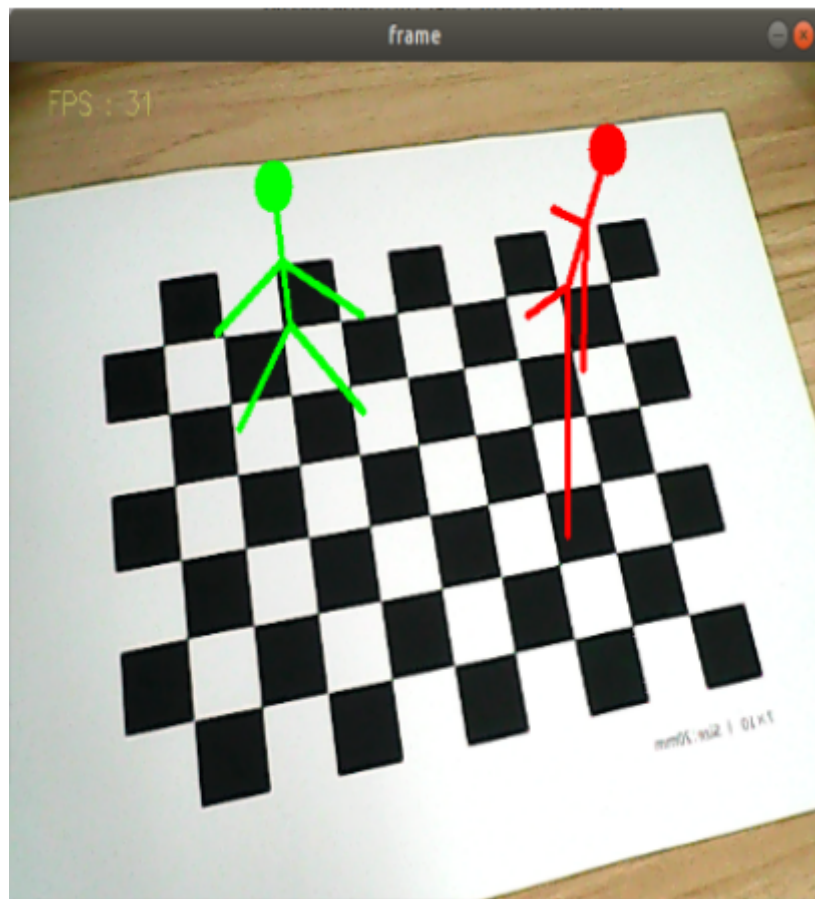
- Use the following command to move the ost.yaml intrinsic parameter file to the configuration file directory

```
tar -zxvf calibrationdata.tar.gz -C
~/yahboomM3_ws/user_ws/src/m3_bringup/config/camera_calibration ost.yaml
```

2.2 Program Startup

Enter in terminal

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=simple_ar
```



After the program starts, place the chessboard in front of the camera. Note that you need to see the entire chessboard, otherwise the program will exit. After recognizing the entire chessboard, the following 12 effects will be displayed,

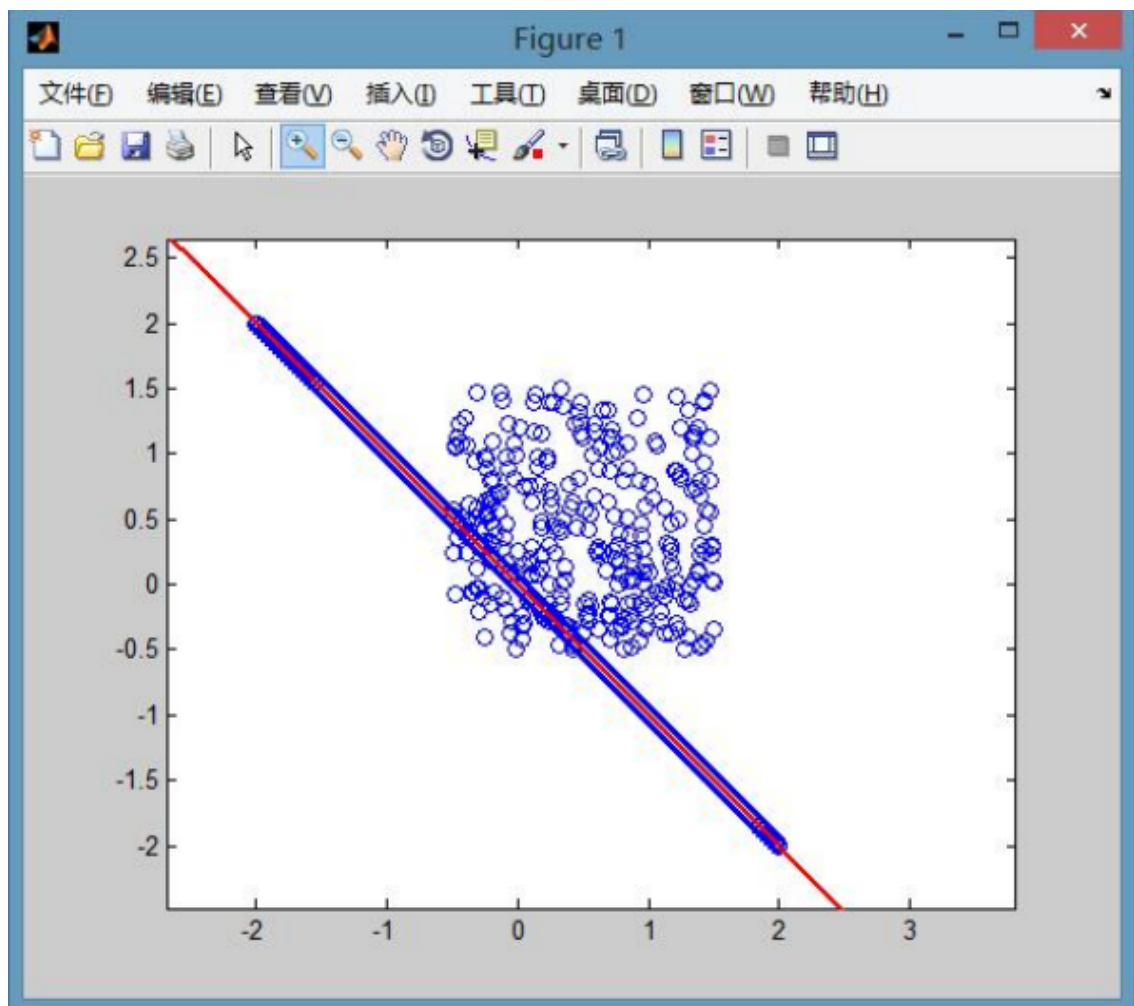
```
[ "Triangle", "Rectangle", "Parallelogram", "WindMill", "TableTennisTable",  
  "Ball", "Arrow", "Knife", "Desk", "Bench", "Stickman", "ParallelBars" ]
```

Click the image and press F to switch the display effects.

2.3 RANSAC Solution

- Algorithm principle: Use RANSAC solution to find object pose from 3D-2D point correspondences

RANSAC algorithm (Random Sample Consensus) was originally a classic algorithm used for data processing. Its purpose is to extract specific components of an object in the presence of large amounts of noise. The following figure illustrates the effect of the RANSAC algorithm. In the figure, some points obviously satisfy a certain line, and another group of points is pure noise. The purpose is to find the line equation in the presence of large amounts of noise, where the amount of noise data is 3 times that of the line.



If using least squares method, such effect cannot be achieved, the line would be slightly above the line in the figure.

- Basic assumptions of RANSAC:
 - Data consists of "inliers", for example: data distribution can be explained by some model parameters;
 - "Outliers" are data that cannot fit the model;
 - Other data belongs to noise.
- Causes of outliers include: extreme values of noise; incorrect measurement methods; wrong assumptions about data. RANSAC also makes the following assumptions: given a set of (usually very small) inliers, there exists a process that can estimate model parameters; and the model can explain or apply to the inliers.

3. Source Code Analysis

Source code location:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/simple_ar.py
```

```
class simple_AR(Node):
    def __init__(self, name):
```

```

super().__init__(name)

self.declare_parameter('rgb_topic', '/ascamera_hp60c/camera_publisher/rgb0/image'
)

self.declare_parameter('yaml_path', '')

rgb_topic=self.get_parameter("rgb_topic").get_parameter_value().string_value
yaml_path =
self.get_parameter("yaml_path").get_parameter_value().string_value

#create pub
self.pub_img = self.create_publisher(Image, '/simpleAR/camera',1)
#create sub
self.sub_graphics =
self.create_subscription(String, '/Graphics_topic',self.choose_Graphics,1)

self.flip = True
self.index = 0
self.graphics = ["Triangle", "Rectangle", "Parallelogram","windMill",
                 "TableTennisTable", "Ball", "Arrow", "Knife", "Desk",
                 "Bench", "Stickman", "ParallelBars"]
self.Graphics = self.graphics[self.index]
self.axis = np.float32([
    [0, 0, -1], [0, 8, -1], [5, 8, -1], [5, 0, -1],
    [1, 2, -1], [1, 6, -1], [4, 2, -1], [4, 6, -1],
    [1, 0, -4], [1, 8, -4], [4, 0, -4], [4, 8, -4],
    [1, 2, -4], [1, 6, -4], [4, 2, -4], [4, 6, -4],
    [0, 1, -4], [3, 2, -1], [2, 2, -3], [3, 2, -3],
    [1, 2, -3], [2, 2, -4], [2, 2, -5], [0, 4, -4],
    [2, 3, -4], [1, 3, -4], [4, 3, -5], [4, 5, -5],
    [1, 2, -3], [1, 6, -3], [5, 2, -3], [5, 6, -3],
    [3, 4, -5], [0, 6, -4], [5, 6, -4], [2, 8, -4],
    [3, 8, -4], [2, 6, -4], [2, 0, -4], [1, 5, -4],
    [3, 0, -4], [3, 2, -4], [0, 3, -4], [1, 2, -4],
    [4, 2, -4], [5, 3, -4], [2, 7, -4], [3, 7, -4],
    [3, 3, -1], [3, 5, -1], [1, 5, -1], [1, 3, -1],
    [3, 3, -3], [3, 5, -3], [1, 5, -3], [1, 3, -3],
    [1, 3, -6], [1, 5, -6], [3, 3, -4], [3, 5, -4],
    [0, 0, -4], [3, 1, -4], [1, 1, -4], [0, 2, -4],
    [2, 4, -4], [4, 4, -4], [0, 8, -4], [5, 8, -4],
    [5, 0, -4], [0, 4, -5], [5, 4, -4], [5, 4, -5],
    [2, 5, -1], [2, 7, -1], [2, 6, -3], [2, 6, -5],
    [2, 5, -3], [2, 7, -3]
])

self.frame = None
self.img_name = 'img'
self.patternSize = (6, 9)
self.bridge = cvBridge()

if os.path.exists(yaml_path):
    # Use Python's yaml library to read ROS format calibration file
    import yaml
    with open(yaml_path, 'r') as file:
        calib_data = yaml.safe_load(file)

    # Extract camera matrix and distortion coefficients from calibration
data

```

```

        camera_matrix_data = calib_data['camera_matrix']['data']
        dist_coeffs_data = calib_data['distortion_coefficients']['data']

        self.cameraMatrix = np.array(camera_matrix_data).reshape(3, 3)
        self.distCoeffs = np.array(dist_coeffs_data)
    else:
        self.distCoeffs, self.cameraMatrix = (), ()
    self.objectPoints = np.zeros((6 * 9, 3), np.float32)
    self.objectPoints[:, :2] = np.mgrid[0:6, 0:9].T.reshape(-1, 2)

    self.subscription = self.create_subscription(
        Image,
        rgb_topic,
        self.image_callback,
        10)

def choose_Graphics(self, msg):
    if not isinstance(msg, String): return
    if msg.data in self.graphics: self.Graphics = msg.data
    else: self.graphics_update()

def graphics_update(self):
    self.index += 1
    if self.index >= len(self.graphics): self.index = 0
    self.Graphics = self.graphics[self.index]

def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    action = cv.waitKey(10) & 0xFF
    frame = self.process(frame, action)
    cv.imshow('frame', frame)
    #if len(binary) != 0: cv.imshow('frame', ManyImgs(1, ([frame, binary])))
    #else: cv.imshow('frame', frame)
    if action == ord('q') or action == 113:
        cv.destroyAllWindows()

def process(self, img, action):
    if self.flip == True: img = cv.flip(img, 1)
    if action == ord('f') or action == ord('F'): self.graphics_update()
    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    # Find the corner of each image
    retval, corners = cv.findChessboardCorners(
        gray, self.patternSize, None,
        flags=cv.CALIB_CB_ADAPTIVE_THRESH + cv.CALIB_CB_NORMALIZE_IMAGE +
cv.CALIB_CB_FAST_CHECK)
    # Find corner subpixels
    if retval:
        corners = cv.cornerSubPix(
            gray, corners, (11, 11), (-1, -1),
            (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001))
    # Compute object pose solvePnP Ransac
    retval, rvec, tvec, inliers = cv.solvePnP Ransac(
        self.objectPoints, corners, self.cameraMatrix, self.distCoeffs)
    # Output image points and Jacobian matrix
    image_points, jacobian = cv.projectPoints(
        self.axis, rvec, tvec, self.cameraMatrix, self.distCoeffs, )

```

```

        img = self.draw(img, corners, image_Points)
        cv.putText(img, self.Graphics, (240, 30), cv.FONT_HERSHEY_SIMPLEX, 0.9,
(0, 0, 255), 1)
        self.pub_img.publish(self.bridge.cv2_to_imgmsg(img, "bgr8"))
        return img

    def draw(self, img, corners, image_Points):
        # drawContours the color order of the drawing is BGR
        img_pts = np.int32(image_Points).reshape(-1, 2)
        if self.Graphics == "Triangle":
            cv.drawContours(img, [np.array([img_pts[14], img_pts[15],
img_pts[23]])], -1, (255, 0, 0), -1)
        elif self.Graphics == "Rectangle":
            cv.drawContours(img, [np.array([img_pts[12], img_pts[13],
img_pts[15], img_pts[14]])], -1, (0, 255, 0), -1)
        elif self.Graphics == "Parallelogram":
            cv.drawContours(img, [np.array([img_pts[12], img_pts[10],
img_pts[15], img_pts[9]])], -1, (65, 105, 225), 1)
        elif self.Graphics == "WindMill":
            cv.drawContours(img, [np.array([img_pts[60], img_pts[38],
img_pts[61], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[10], img_pts[14],
img_pts[58], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[62], img_pts[63],
img_pts[23], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[25], img_pts[64],
img_pts[65], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.line(img, tuple(img_pts[64]), tuple(img_pts[35]), (0, 255, 0), 3)
        elif self.Graphics == "TableTennisTable":
            cv.line(img, tuple(img_pts[0]), tuple(img_pts[60]), (255, 0, 0), 3)
            for i in range(1, 4):
                cv.line(img, tuple(img_pts[i]), tuple(img_pts[65 + i]), (255, 0,
0), 3)
            cv.drawContours(img, [np.array([img_pts[60], img_pts[66],
img_pts[67], img_pts[68]])], -1, (0, 255, 0), -1)
            cv.drawContours(img, [np.array([img_pts[23], img_pts[69],
img_pts[71], img_pts[70]])], -1, (0, 0, 255), -1)
        elif self.Graphics == "Ball": cv.circle(img, tuple(img_pts[22]), 30, (0,
0, 255), -1)
        elif self.Graphics == "Arrow":
            cv.drawContours(img, [np.array([img_pts[13], img_pts[34],
img_pts[36]])], -1, (0, 255, 0), -1)
            cv.drawContours(img, [np.array([img_pts[37], img_pts[15],
img_pts[10], img_pts[38]])], -1, (0, 255, 0), -1)
        elif self.Graphics == "Knife":
            cv.drawContours(img, [np.array([img_pts[58], img_pts[24],
img_pts[35], img_pts[47]])], -1, (160, 252, 0),
-1)
            cv.drawContours(img, [np.array([img_pts[40], img_pts[38],
img_pts[21], img_pts[41]])], -1, (30, 144, 255),
-1)
            cv.drawContours(img, [np.array([img_pts[42:46]])], -1, (0, 0, 255),
-1)
        elif self.Graphics == "Desk":
            for i in range(4):
                cv.line(img, tuple(img_pts[4 + i]), tuple(img_pts[12 + i]),
(163, 148, 128), 3)

```



```

        cv.drawContours(img, [np.array([img_pts[14], img_pts[12],
img_pts[13], img_pts[15]])], -1, (0, 199, 140),
                        -1)
    elif self.Graphics == "Bench":
        for i in range(4):
            cv.line(img, tuple(img_pts[48 + i]), tuple(img_pts[52 + i]),
(255, 0, 0), 3)
            cv.drawContours(img, [img_pts[52:56]], -1, (0, 0, 255), -1)
            cv.drawContours(img, [img_pts[54:58]], -1, (139, 69, 19), -1)
    elif self.Graphics == "Stickman":
        cv.line(img, tuple(img_pts[18]), tuple(img_pts[4]), (0, 0, 255), 3)
        cv.line(img, tuple(img_pts[18]), tuple(img_pts[6]), (0, 0, 255), 3)
        cv.line(img, tuple(img_pts[18]), tuple(img_pts[21]), (0, 0, 255), 3)
        cv.line(img, tuple(img_pts[21]), tuple(img_pts[19]), (0, 0, 255), 3)
        cv.line(img, tuple(img_pts[21]), tuple(img_pts[20]), (0, 0, 255), 3)
        cv.line(img, tuple(img_pts[21]), tuple(img_pts[22]), (0, 0, 255), 3)
        cv.circle(img, tuple(img_pts[22]), 15, (0, 0, 255), -1)
        cv.line(img, tuple(img_pts[74]), tuple(img_pts[72]), (0, 255, 0), 3)
        cv.line(img, tuple(img_pts[74]), tuple(img_pts[73]), (0, 255, 0), 3)
        cv.line(img, tuple(img_pts[74]), tuple(img_pts[37]), (0, 255, 0), 3)
        cv.line(img, tuple(img_pts[37]), tuple(img_pts[76]), (0, 255, 0), 3)
        cv.line(img, tuple(img_pts[37]), tuple(img_pts[77]), (0, 255, 0), 3)
        cv.line(img, tuple(img_pts[37]), tuple(img_pts[75]), (0, 255, 0), 3)
        cv.circle(img, tuple(img_pts[75]), 15, (0, 255, 0), -1)
    elif self.Graphics == "ParallelBars":
        for i in range(4):
            cv.line(img, tuple(img_pts[4 + i]), tuple(img_pts[12 + i]),
(255, 0, 0), 3)
            cv.line(img, tuple(img_pts[8]), tuple(img_pts[9]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[10]), tuple(img_pts[11]), (0, 0, 255), 3)
    return img

```

Program flow chart,

